

Introduction to *NLTK*

Inf2a Labs 3 and 4

Ivan V. Meza-Ruiz, Srinivasan C Janarthanam, Bonnie Webber
and Chris Gorgolewski

16 October 2009

NLTK is a suite of Python libraries and programs for symbolic and statistical natural language processing (<http://www.nltk.org/>). It has extensive documentation – in particular, in the recently published book *Natural Language Processing with Python*, available on-line at <http://www.nltk.org/book>. Each section of this lab sheet points to relevant section(s) of the book.

▷ *Open a terminal window, and go to your MyPython folder.*

▷ *Launch the Python shell.*

1 Starting with *NLTK*

To load the *NLTK* libraries we use `import` statement. The `book` module contains all the material needed for this tutorial.

▷ *Type:*

```
>>> from nltk.book import *
>>> text1
>>> text2
```

If we use the equally correct command

```
>>> import nltk.book
```

how would you have to refer to `text1` and `text2`?

The description of a class is available using the `help` function. (To finish with help mode, type `q`.)

▷ *Type:*

```
>>> help(text1)
```

The documentation of the *NLTK* API covers all available modules and classes: <http://nltk.googlecode.com/svn/trunk/doc/api/index.html>

2 Tokenization

For most kinds of linguistic processing, we need to identify and classify the words of the text in terms of the part-of-speech they play each time they occur. Here we'll see why this can be a non-trivial task. First, we'll try dividing a text string into tokens using Python's built-in method `split`.

▷ *Type:*

```
>>> text = 'Hello, World! This is a test string.'
>>> text.split()
```

`split` breaks a string at each white space. As you can see, tokenization based on whitespace alone is not enough to isolate word tokens. (What shows this?) Instead, we can use the method `tokenize.regex_tokenize`. This uses a regular expression to specify how text should be split up. This regular expression specifies the characters that can be included in a valid word.

▷ *Type:*

```
>>> from nltk import tokenize
>>> text = "Hello. Isn't this fun?"
>>> pattern = r'\w+|[\^\w\s]+'
>>> tokenize.regex_tokenize(text, pattern)
```

The regular expression assigned above to `pattern` – that is, `r'\w+|[\^\w\s]+'` – specifies that a valid word can be formed from a sequence of “word” characters, i.e., characters other than whitespace or punctuation (the subexpression `\w+`), or characters that are neither word characters nor whitespace characters (the subexpression `[\^\w\s]+'`, where `^` means complement).

While this pattern succeeds in removing final punctuation, it does not deliver correct results with strings containing amounts such as *\$22.40* and *44.50%* which either start or end with a non-word character that isn't punctuation and contain medial non-word characters.

▷ *Type:*

```
>>> text = 'That poster costs $22.40.'
>>> tokenize.regex_tokenize(text, pattern)
```

Here we might want to keep the symbol `$` and `%` attached to its end of number and the symbol `.` attached in its middle. The union of the previous regular expression with this one: `\$\d+\.\d+|[\^\w\s]+'`, where `\d` represents a decimal digit, solves the problem of initial `$` and medial `..`

▷ *Type:*

```
>>> pattern = r'\w+|\$\d+\.\d+|[\^\w\s]+'
>>> tokenize.regex_tokenize(text, pattern)
```

There is more information about regular expressions in Python at:

<http://www.python.org/doc/2.3/lib/re-syntax.html>.

Exercise 1

Add to `pattern` (as defined in the previous code snippet) the option of tokenizing a percentage as a single token, as in the following three tokens:

- 100.00%, 10.5%, and 10.234%.

Use the union operator (`|`) to add your option to the start of `pattern`. Test with the following text:

```
>>> text = 'The interest does not exceed 8.25%.'
```

In dealing with text from technical fields such as biology or chemistry, often the first thing one has to do is customize the tokenisation pattern to deal with the special notation in the field. Here is an example from biology, which has a special notation for specifying chromosomal locations:

The CFTR gene is located on the long arm of chromosome 7 at position 7q31.2.

<http://ghr.nlm.nih.gov/handbook/illustrations/chromosomallocation>

Test how the current regular expression tokenizes this location specification. Also test how it tokenizes URLs: This is another case where further customization may be needed. There is more about tokenization in the NLTK book, Section 3.7 (<http://nltk.googlecode.com/svn/trunk/doc/book/ch03.html>).

3 Tokens, Types and Frequencies

`nltk.book` contains several texts from the Gutenberg corpus, including *Moby Dick* (`text1`), *Monty Python and the Holy Grail* (`text6`), and a corpus of personal ads (`text8`). Let's see the contents of `text1`.

▷ *Type:*

```
>>> from nltk.book import *
>>> list(text1)
```

`text1` is a list of word and punctuation tokens, so we can use Python's built-in `len` function to determine its length in terms of words and punctuation marks. This count gives the number of **Tokens**.

▷ *Type:*

```
>>> len(text1)
```

This count includes words (and punctuation marks) that may repeat several times in the text. How do we get a list of distinct words and their count? This is called a **Type** count (because it's the number of word types).

▷ *Type:*

```
>>> set(text1)
>>> len(set(text1))
```

The *lexical richness* of a given text is a count, on average, of how many times each word appears in a text. We get this by dividing the token count by the type count.

▷ *Type:*

```
>>> len(text1) / len(set(text1))
```

Recall that the operator `/` gives integer division if both its operands are integers, so float the numerator to get a precise figure.

▷ *Type:*

```
>>> float(len(text1)) / len(set(text1))
```

The inverse of the *lexical richness* of a text is its *type-token ratio*, which is a standard measure of vocabulary variation within a text. (The more types in comparison with the number of tokens in a text, the greater its lexical variety.) Compare the *type-token ratio* of *Moby Dick* (`text1`), *Monty Python and the Holy Grail* (`text6`), and the corpus of personal ads (`text8`). Which of the three uses the largest variety of different words?

Now we will consider how to find the most frequent words in a text using the *NLTK* module `FreqDist`.

▷ *Type:*

```
>>> from nltk import FreqDist
>>> fd = FreqDist(text1)
```

To find the frequency (ie, the number of individual tokens) of a given type (e.g. 'the'), use `fd[type]`.

▷ *Type:*

```
>>> fd['the']
>>> fd['with']
```

If `fd` is a frequency distribution, you can get the list of its samples (ie, types) in decreasing order of frequency using `fd.keys()`. Since this will produce a very long list, you might want to focus on the first 20 or 30 items. The function `fd.items()` will give you both the samples and their frequencies, again in decreasing order.

▷ *Type:*

```
>>> fd.keys()
>>> fd.keys()[:30]
>>> fd.items()[:30]
```

Exercise 2

- Find the token count, type count, lexical richness and top 20 most frequent types in `text2`, `text6` and `text8`.
- This process counts tokens like `The` and `the` as two different types, while they are really the same. Use `s.lower()` (where `s` is a string) to normalize the words to lower case before counting for types and tokens.

Whereas a frequency distribution gives the distribution of word types across the entire text, a conditional frequency distribution gives the distribution relative to a particular context – e.g., the distribution of words found after the verb “said”. We can use `ConditionalFreqDist` to determine this. First, we pass each token of the corpus together with its previous token to the `ConditionalFreqDist` class. We call this process *training*.

▷ *Type:*

```
>>> from nltk import ConditionalFreqDist
>>> cfdist = ConditionalFreqDist()

>>> prev=None
>>> for token in text1:
...     lcase = token.lower()
...     cfdist[prev].inc(lcase)
...     prev = lcase
...
```

Notice that the first token of the corpus has no “previous word”, so here we use `None` as the context for the first token. Each `cfdist[prev]` is a frequency distribution, and so we can use `cfdist[prev].inc(token)` function to increment the count for the sample `token` in the distribution. We can see the distribution of words which follow a given word using the `keys()` method.

▷ *Type:*

```
>>> cfdist['said'].keys()
```

Try this with a different word of your own choice.

We can use the information stored in `ConditionalFreqDist` to create a text generator using the most frequent word following a given word. The following code creates a sentence of word length 20 starting with ‘he’.

▷ *Type:*

```
>>> word = 'an'
>>> for i in range(20):
...     print word,
...     word = cfdist[word].max()
...
```

See what sentence you get if you start with a different word.

Exercise 3

1. For many words, the text generator gets stuck in a cycle very quickly. Modify the program to choose the next word randomly from a list of the words following the given word. (Hint: get the word list `lwords` using `keys()` method, and then randomly choose a word from the list using `random.choice(lwords)`. Don't forget to `import` the `random` module.)
2. The above program has been trained on `text1` of the Gutenberg corpus. Now, train your system on some other texts from the Gutenberg corpus and use each to generate random text following the initial word `an` or from one of the words you chose yourself.
3. Try using the most frequent word in the text as the start word of each of your generators. (Recall that `FreqDist` returns an absolute, rather than a conditional, frequency distribution on a set of samples – here, the words in a text.)

Frequency distributions are discussed in Section 1.3 of the NLTK book (<http://nltk.googlecode.com/svn/trunk/doc/book/ch01.html>) and conditional frequency distributions, in Section 2.2 (<http://nltk.googlecode.com/svn/trunk/doc/book/ch02.html>).

4 Tagged corpora

NLTK is distributed with corpora that can be accessed using `nltk.corpus` package. First we import the Brown Corpus, the oldest million-word, part-of-speech tagged electronic corpus of English. Each of the sections *ca* through *cr* represents a different *genre*, or category of text. A list of these categories can be accessed using `categories()`, and a list of the files in each section can be accessed using `fileids()`.

▷ *Type:*

```
>>> from nltk.corpus import brown
>>> help(brown)
>>> brown.fileids()
>>> brown.categories()
>>> brown.fileids(['adventure', 'romance'])
>>> brown.categories(['ca01', 'cp28'])
```

The methods `words` and `sents` provide access to the corpus as a list of words or a list of sentences, respectively.

▷ *Type:*

```
>>> brown.words('ca01')
>>> brown.sents('ca01')
>>> brown.sents('ca01')[0]
>>> brown.sents(brown.fileids(['adventure']))
```

Try the example with 'cb01' and another category from the corpus.

With a tagged corpus like the Brown Corpus, the methods `tagged_words` and `tagged_sents` provide access to its tagged text. Every word in the sentence is tagged with a PoS tag and is presented as a list of binary tuples containing the word and its tag (for e.g. ('The', 'AT')).

▷ *Type:*

```
>>> print brown.tagged_words('ca01')
>>> print brown.tagged_sents('ca01')
>>> print brown.tagged_sents('ca01')[0]
>>> print brown.tagged_words(categories=romance)[:50]
```

Since the structures accessed through `nltk.corpus` are basic Python structures, we can use basic Python to analyse the corpus. For instance, we can count the number of times each part-of-speech tag occurs in the Brown corpus.

▷ *Type:*

```
>>> dict={}
>>> for sentc in brown.tagged_sents('ca01'):
...     for word,tag in sentc:
...         if tag in dict:
...             dict[tag]+=1
...         else:
...             dict[tag]=1
...
>>> for tag,count in dict.iteritems():
...     print tag, count
...
```

Exercise 4

Create a dictionary of the words used in item 'ca01' of the Brown corpus. The key of the dictionary should be the part-of-speech tag of a word. The value of the dictionary should be the list of the words which are tagged with the part-of-speech tag. For instance, `dict['NN']` returns ['director', 'chairman...'], if *directory* and *chairman* were labelled as 'NN'. Test the dictionary using a similar `for` loop of the previous example.

Corpora rarely come already PoS-tagged. In NLTK, there are several ways to build PoS taggers. A simple way to tag is to assign the same tag to all the words in the sentence. Using `DefaultTagger`, we can build such a tagger. The following tagger tags all words with NN tag.

▷ *Type:*

```
>>> from nltk import DefaultTagger
>>> tokens = 'John saw 3 polar bears .'.split()
>>> default_tagger = DefaultTagger('NN')
>>> default_tagger.tag(tokens)
```

But, how good is this tagger? Using `evaluate()`, we can find out how accurate the tagger is with respect to the `brown` corpus. (The description that comes from `help(UnigramTagger)` shows that `evaluate` applies the tagger to the untagged words in a (tagged) corpus and then scores the tagger's performance against the gold standard tags in that corpus.)

▷ *Type:*

```
>>> default_tagger.evaluate(brown.tagged_sents('ca01'))
```

Can we build a better tagger? We don't want to tag the verb `said` with `NN`. Using `UnigramTagger`, we can build a tagger that assigns word in the training data its most frequent tag within that corpus. This tagger will assign more appropriate tags to the words (for e.g. `VBD` to `said`) than the `DefaultTagger`.

▷ *Type:*

```
>>> from nltk import UnigramTagger
>>> tokens = 'John saw three large cats and 14 dogs'.split()
>>> unigram_tagger1 = UnigramTagger(brown.tagged_sents(categories='news'))
>>> unigram_tagger2 = UnigramTagger(brown.tagged_sents(categories='romance'))
>>> unigram_tagger1.tag(tokens)
>>> unigram_tagger2.tag(tokens)
>>> unigram_tagger2.evaluate(brown.tagged_sents(categories='romance'))
>>> unigram_tagger1.evaluate(brown.tagged_sents(categories='romance'))
```

`UnigramTagger` will ignore the contextual information around the words. For instance, `frequent` will always be tagged as an adjective (`JJ`), even if it not an adjective according to the context (for e.g. `I frequent this cafe`). To take account of the previous word as context, NLTK provides a bigram tagger. Such a tagger can learn that the word “frequent” following “I” should be tagged as a verb (eg, “I frequent this cafe”), while the word “frequent” following “this” should be tagged as an adjective (eg, “this frequent annoyance”). A bigram tagger can out-perform a unigram tagger, if sufficient data is provided to train it. There is more about bigram taggers and other taggers in Section 5.4 of the NLTK book chapter on *Categorizing and Tagging Words* <http://nltk.googlecode.com/svn/trunk/doc/book/ch05.html>.

NLTK's currently recommended tagger (already trained) is called `pos_tag`. It can be imported from NLTK.

5 Context-free Grammars

In NLTK, you can either use the module `parse_cfg` to define a context-free grammar, or create and edit a grammar in a text file and then load it into NLTK using `nltk.data.load('file:<filename>.cfg')` (For the latter, make sure you put a `.cfg` suffix on the filename.) Focussing here on `parse_cfg`, by convention, the left-hand side of the first production is taken to be the start symbol, and every symbol that occurs on the LHS of a rule is a non-terminal symbol and every one that doesn't is a terminal.

▷ *Type:*

```
>>> from nltk import parse_cfg
>>> grammar2 = parse_cfg('''
...     S -> NP VP
...     NP -> "I" | "John" | "Mary" | "Bob" | Det N
...     VP -> V NP | V NP PP
...     V -> "saw" | "ate"
...     Det -> "a" | "an" | "the" | "my"
...     N -> "dog" | "telescope" | "apple"
...     PP -> P NP
...     P -> "on" | "with"
...     ''')

>>> grammar2

>>> grammar2.start()

>>> grammar2.productions()
```

For a grammar, we can parse a sentence and get its syntactic tree using the `parse.RecursiveDescent()` function.

▷ *Type:*

```
>>> from nltk import parse
>>> sntc1 = 'I saw Mary'.split()
>>> sntc2 = 'John ate my apple'.split()
>>> rd_parser = parse.RecursiveDescentParser(grammar2)
>>> for p in rd_parser.nbest_parse(sntc1):
...     print p

>>> for p in rd_parser.nbest_parse(sntc2):
...     print p
```

Exercise 5

Create a new grammar (`grammar3`) that consists of `grammar2` plus the production `NP -> Det N PP`.

1. Parse the sentence *Mary saw the dog with the telescope*. How many parse trees do you get?
2. For `grammar3`, write down two unambiguous sentences (ie, sentences which have a single syntactic tree structure with respect to the grammar), two ambiguous sentences (each of which yields more than one tree), and one ungrammatical sentences (that yields no tree at all).

6 Treebank

NLTK also includes a 10% fragment of the Wall Street Journal section of the Penn Treebank. Each sentence of the corpus is available in three forms; (1) as tokenized text, (2) as tokens labelled with part of speech, and (3) as parse trees. These can be accessed using `treebank.raw()` for the raw text, `treebank.tagged_sents()` for the tagged text, and `treebank.parsed_sents()` for the parse trees.

▷ *Type:*

```
>>> from nltk.corpus import treebank
>>> help(treebank)
>>> treebank.fileids()

>>> print treebank.raw('wsj_0001.mrg')

>>> print treebank.tagged_words('wsj_0001.mrg')[:20]

>>> print treebank.tagged_sents('wsj_0001.mrg')[0]

>>> print treebank.parsed_sents('wsj_0001.mrg')[0]
```

The argument in the above three functions, `'wsj_0001.mrg'`, is a section off the treebank. (As shown above, we can see the list of all sections using `treebank.fileids()`). The function `treebank.parsed_sents() [0]` returns an object of class `nltk.Tree`. We can analyse the tree using different functions available in this class.

▷ *Type:*

```
>>> t = treebank.parsed_sents('wsj_0001.mrg')[0]
>>> t.node

>>> t.leaves()

>>> print t

>>> len(t)

>>> t[0]

>>> t[0].node

>>> len(t[0])
```

`t.node` contains the top-most node in the tree `t`. We can see the list of words that yields the parse tree using `t.leaves()`. Using `len(t)`, we can get the number of child nodes to `t.node`. Each of these child nodes and their subtrees can be accessed using `t[0].node` and `t[0]` respectively.

Using the above utilities, we can locate the subject of any given sentence. The following code prints the subject NP phrase in the first sentence of 'wsj_0001.mrg'

▷ *Type:*

```
>>> t = treebank.parsed_sents('wsj_0001.mrg')[0]
>>> for ch_tree in t:
...     if (ch_tree.node.startswith('NP-SBJ')):
...         print ch_tree.leaves()
```

Exercise 6

1. Extend the above program to identify the subject in all the sentences in 'wsj_0003.mrg'.
2. A subordinate clause in a sentence will have its own subject. So, extend the code (using recursion) from the previous problem to identify all the subjects in every sentence.

7 IPython

Thanks to its extensive high level API and ease of coding, Python is an excellent tool for prototyping and quickly implementing new ideas. However, the Python shell leaves a lot of space for improvement. IPython (the Interactive Python shell) fills this gap with several improvements and additional features. To start working with IPython, simply type `ipython` instead of `python`. IPython gives you the following improvements:

API documentation If you don't remember what a particular function, object or IPython command does, you can use the `?` syntax to find out. Just suffix the function with `?` to get all its properties and its **docstring** (usually describing how to use it).

```
In [1]: import re
In [2]: re.match?
```

Tab completion Each time you start typing and press **TAB**, IPython will try to complete the syntax or give you possible options. This includes local variables that you have already declared (which you can list using `who` and `whos` — try them out!), as well as built-in functions and methods. It can even complete `from package_name import ...` statements. This is very handy when you don't remember the API.

```
In [1]: from re import s
search      split      sre_compile  sre_parse   sub
subn        sys
```

Session storing and logging Often you test some simple procedures in Python console, before you write a script. It would be very handy to have a record of all the steps towards the final solution that would be the basis of the

script. IPython provides you with simple but useful logging system. Check out the `logstart`, `logon` and `logoff` commands. IPython allows you also to store some variables so they would be available each time you load the console — you can do it using the `store` command. This is handy if you use IPython for calculations. You have also probably noticed that every line in IPython starts with an `Out[]` or `In[]` prefix. You can use them to access previous commands and there results as you would use any other variable. To re-execute a command, use the `exec` command.

```
In [1]: logstart
Activating auto-logging. Current session state plus future input saved.
Filename      : ipython_log.py
Mode          : rotate
Output logging : False
Raw input log  : False
Timestamping  : False
State         : active
In [2]: logstop
```

Using external editor To quickly draft a class or just write a longer piece of code in an external editor, use the `edit` command. This will run the editor defined by the `$EDITOR` environment variable on a temporary file and execute it in the current context after you close the editor. To edit the same bit again (in case you made a mistake) use the `edit -p` command. Temporary files are stored in the `tmp` folder but you can explicitly save them to your local folder using the editor.

```
In [1]: edit -p
IPython will make a temporary file named: /tmp/ipython_edit_pWwSyO.py
Editing... done. Executing edited code...
Out[1]: ~'class Greeter:\n\tdef __init__(self, greeting):\n\t\tself.greeting = gre
```

Running scripts If you work on external script or module, each time you make changes to it you have to either run it through `python -i` or if it is just a module import it (or reload it). IPython introduces a helpful command `run module_name` which you can use from within IPython. It will load the content of the specified file or module into the current namespace.

```
In [1]: run greeter.py
```

Autocall If you are tired of typing parentheses, IPython allows you to omit them when calling simple functions. For example, IPython will interpret `str 42` as `str(42)`¹.

```
In [1]: str 42
-----> str(42)
Out[1]: '42'
```

IPython provides more useful commands (it even supports debugging), which are all documented on the website of the project at <http://ipython.scipy.org>.

¹This feature might be slightly confusing when you use `print` command. `print string1, string2` in IPython will be converted into `print(string1, string2)` which for python up to version 3.0 is something different (first one prints a concatenation and the second a tuple). You can turn off autocall by passing the `-autocall 0` argument